Задание к лабораторной работе №3

«Автодополнение кода: LSTM и GRU (NumPy) на корпусах Python/C++/Java

+ дополнительное задание с BiLSTM»

Содержание

1 Общие требования **2**

1.1 Архитектуры 2

1.2 Требования к реализации 2

1.3 Обязательные результаты 3

2 Данные и постановка задачи **4**

2.1 Структура датасета 4

2.2 Постановки задач 4

2.3 Формат `dataset_char.npz` 6

3 Варианты и рекомендуемые параметры **7**

3.1 Выбор варианта 7

3.2 Рекомендуемые гиперпараметры (для всех вариантов) 7

3.3 Что сдавать 7

1 Общие требования

1.1 Архитектуры

Во всех вариантах реализуются и сравниваются две рекуррентные архитектуры **на NumPy** без использования специализированных библиотек глубокого обучения (PyTorch, TensorFlow, Keras и аналогов). Допускаются: `numpy`, `math`, `random`, `matplotlib`, `sklearn.metrics` (только для вычисления метрик).

LSTM (Long Short-Term Memory)

Входные данные на шаге t: $x_t \in \mathbb{R}^d$; скрытое состояние $h_t \in \mathbb{R}^H$; состояние ячейки $C_t \in \mathbb{R}^H$. Начальные условия: $h_0 = \mathbf{0},\ C_0 = \mathbf{0}$.

$$f_t = \sigma(W_f\, x_t + U_f\, h_{t-1} + b_f), \tag{1}$$

$$i_t = \sigma(W_i\, x_t + U_i\, h_{t-1} + b_i), \tag{2}$$

$$\tilde{C}_t = \tanh(W_C\, x_t + U_C\, h_{t-1} + b_C), \tag{3}$$

$$o_t = \sigma(W_o\, x_t + U_o\, h_{t-1} + b_o), \tag{4}$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \tag{5}$$

$$h_t = o_t \odot \tanh(C_t). \tag{6}$$

Здесь $\sigma(x) = 1/(1+e^{-x})$ — сигмоида; $\odot$ — поэлементное произведение. Матрицы весов: $W_f, W_i, W_C, W_o \in \mathbb{R}^{H\times d}$, $U_f, U_i, U_C, U_o \in \mathbb{R}^{H\times H}$; смещения $b_f, b_i, b_C, b_o \in \mathbb{R}^H$.

GRU (Gated Recurrent Unit)

$$r_t = \sigma(W_r\, x_t + U_r\, h_{t-1} + b_r), \tag{7}$$

$$z_t = \sigma(W_z\, x_t + U_z\, h_{t-1} + b_z), \tag{8}$$

$$\tilde{h}_t = \tanh\big(W_h\, x_t + U_h\, (r_t \odot h_{t-1}) + b_h\big), \tag{9}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \tag{10}$$

Матрицы весов: $W_r, W_z, W_h \in \mathbb{R}^{H\times d}$, $U_r, U_z, U_h \in \mathbb{R}^{H\times H}$.

1.2 Требования к реализации

1. Реализовать forward pass и BPTT для обеих архитектур вручную на `numpy`. Использование автоматического дифференцирования (autograd, PyTorch backward и аналоги)

запрещено.

2. Инициализация весов: равномерная $\mathcal{U}(-k,\ k)$, где $k = 1/\sqrt{H}$; смещения инициализировать нулями.

3. Оптимизация: **Adam**. Параметры Adam и gradient clipping требуется явно указать в отчёте.

4. Разбивка корпуса на train / val / test выдаётся готовой в составе варианта и не изменяется.

5. Обучение вести до лучшего результата на валидации, но не более N эпох, где N указано в варианте. В отчёте необходимо указать эпоху, на которой достигнут лучший результат на validation.

1.3 Обязательные результаты

1. **Графики обучения** (отдельно для LSTM и GRU):

   - Perplexity (train и val) по эпохам;
   - Top-k accuracy (val) по эпохам для $k \in \{1, 5, 10\}$.

2. **Сравнительная таблица** финальных метрик (по лучшей эпохе на val): Perplexity и Top-k accuracy на val и test для LSTM и GRU.

3. **Демонстрация автодополнения:** привести минимум 10 примеров продолжения по префиксу: для каждого префикса вывести Top-10 вариантов следующего символа с вероятностями.

2 Данные и постановка задачи

2.1 Структура датасета

Вместе с заданием выдаётся 30 вариантов датасетов для трёх языков программирования: **Python**, **C++** и **Java**. Датасеты лежат в каталоге:

`datasets_code_autocomplete/{python,cpp,java}/variant_XX/`.

Варианты отличаются обучающим множеством (`train`), а валидационное и тестовое множества (`val`, `test`) являются общими для всех вариантов одного языка.

Внутри каждого варианта находятся:

- `data_char/dataset_char.npz` — подготовленная версия для обучения (см. ниже). **Это основной формат, используемый в лабораторной работе.**
- `data_char/meta.json` — параметры подготовки (размер словаря и ограничения на выборку).

2.2 Постановки задач

Автодополнение следующего символа (LSTM/GRU)

Задача формулируется как character-level языковая модель: по последовательности символов $c_1, \ldots, c_T$ предсказать следующий символ на каждом шаге c_{t+1}.

Используется обучаемая матрица эмбеддингов символов $E \in \mathbb{R}^{|V| \times d}$, где $|V|$ — размер словаря символов. На вход сети подаются индексы символов, эмбеддинги выбираются как строки из E, после чего подаются в LSTM/GRU.

Выходной слой предсказывает распределение вероятностей следующего символа:

$$\hat{y}_t = \mathrm{softmax}\big(W_y\, h_t + b_y\big), \quad W_y \in \mathbb{R}^{H \times |V|}, \quad b_y \in \mathbb{R}^{|V|}.$$

Функция потерь: средняя cross-entropy по всем шагам и примерам в mini-batch:

$$\mathcal{L} = -\frac{1}{BT} \sum_{b=1}^{B} \sum_{t=1}^{T} \log \hat{p}\Big(c_{t+1}^{(b)} \mid c_1^{(b)}, \ldots, c_t^{(b)}\Big)\,.$$

Основная метрика: **Perplexity** = $\exp(\mathcal{L})$. Дополнительные метрики: **Top-k accuracy** для $k \in \{1, 5, 10\}$.

Восстановление символа в середине (BiLSTM)

BiLSTM использует двусторонний контекст (слева и справа), поэтому для автодополнения "следующего" символа эта архитектура не является корректной в онлайновом сценарии. Вместо этого BiLSTM применяется для задач, где правый контекст доступен.

Постановка (Fill-in-the-Middle). Для каждого обучающего окна длины T выбирается позиция m (рекомендуется фиксировать $m = \lfloor T/2 \rfloor$). Символ c_m считается "пропущенным" и должен быть предсказан по контексту слева и справа.

Использовать следующую схему:

- вход: последовательность длины T, в которой на позиции m стоит специальный токен `<MASK>`;
- цель: исходный символ c_m;
- модель: два прохода LSTM по одной и той же последовательности: прямой (слева направо) и обратный (справа налево); затем на позиции m объединить состояния конкатенацией и предсказать распределение символов;
- функция потерь: cross-entropy **только для позиции** m (иначе получится другая задача, не "восстановление" в одной точке).

Как вводится `<MASK>`. Пусть базовый словарь символов из `dataset_char.npz` имеет размер $|V|$. Для этого задания добавить один дополнительный токен:

$$\texttt{mask_id} = |V|.$$

Новый размер словаря становится $|V| + 1$. Эмбеддинг `<MASK>` обучается (добавляется как новая строка в E).

Perplexity и accuracy. Так как loss считается только по одной позиции, perplexity определять как $\exp(\mathcal{L}_{\text{mask}})$, где $\mathcal{L}_{\text{mask}}$ — средняя cross-entropy для предсказания символа в позиции m. Top-k accuracy считать аналогично: попадание истинного c_m в Top-k предсказаний.

Рекомендованные параметры. Начать с тех же T, B, d, что и для LSTM/GRU. Рекомендуется взять H меньше, например $H = 64$, так как BiLSTM после конкатенации даёт размер признаков $2H$. Ограничение по эпохам и early stopping — те же, что в основной части.

Что включить в отчёт по BiLSTM.

- графики обучения (val perplexity и val top-k по эпохам);
- финальные метрики на val и test (по лучшей эпохе на validation);
- минимум 10 примеров восстановления: показать окно с `<MASK>` и Top-10 предсказаний с вероятностями.

2.3 Формат `dataset_char.npz`

Архив `dataset_char.npz` содержит:

- `vocab` — массив кодпоинтов (Unicode codepoint) символов словаря; индекс 0 зарезервирован под `<UNK>`.
- `train`, `val`, `test` — 1D массивы индексов символов (плоские “потоки”), полученные конкатенацией всех фрагментов соответствующего сплита.

Обучение строится через выбор случайных окон фиксированной длины T из потока:

$$x = (c_1, \ldots, c_T), \qquad y = (c_2, \ldots, c_{T+1}).$$

Загрузка датасета (пример кода)

```
import numpy as np

npz = np.load("data_char/dataset_char.npz", allow_pickle=True)
vocab_codepoints = npz["vocab"]          # uint32, vocab[0]=0 -> <UNK>
train = npz["train"].astype(np.int64)    # 1D token ids
val   = npz["val"].astype(np.int64)
test  = npz["test"].astype(np.int64)

def id_to_char(i: int) -> str:
    if i == 0:
        return "?"  # <UNK>
    return chr(int(vocab_codepoints[i]))

print("vocab_size:", len(vocab_codepoints))
print("train_tokens:", len(train))
print("sample text:", "".join(id_to_char(i) for i in train[:300]))
```

3 Варианты и рекомендуемые параметры

3.1 Выбор варианта

Вариант определяется номером `XX`. Язык выбирается преподавателем или по согласованию. Данные варианта находятся в каталоге `datasets_code_autocomplete/<lang>/variant_XX/`, где `<lang>` $\in$ {`python`, `cpp`, `java`}.

3.2 Рекомендуемые гиперпараметры (для всех вариантов)

Рекомендуется использовать следующие настройки (при необходимости допускается подстройка, но изменения должны быть обоснованы):

- длина окна (sequence length): $T = 128$ (допустимо 96–256);
- batch size: $B = 128$ (допустимо 64–256);
- оптимизатор Adam: $\eta = 2 \cdot 10^{-3},\ \beta_1 = 0.9,\ \beta_2 = 0.999,\ \varepsilon = 10^{-8}$;
- gradient clipping: **5.0**;
- embedding dim: $d = 24$ (допустимо 16–64);
- hidden size: $H = 96$ (допустимо 64–256);
- максимум эпох: $N = 20$, early stopping patience: **6**;
- steps per epoch: **120**; eval batches: **20**.

3.3 Что сдавать

В отчёте необходимо явно указать:

- номер варианта (`variant_XX`);
- какие файлы использовались (`data_char/dataset_char.npz`);
- все использованные гиперпараметры.